

Autonomous Optimization of Business Processes

Markus Kress and Detlef Seese

Institute AIFB, Karlsruhe Institute of Technology (KIT)
University Karlsruhe (TH), 76128 Karlsruhe, Germany
{kress,seese}@aifb.uni-karlsruhe.de
<http://www.aifb.uni-karlsruhe.de>

Abstract. In this paper we introduce the intelligent Executable Product Model (iEPM) approach for the autonomous optimization of service industry's business processes. Instead of using a process model, we use an Executable Product Model (EPM). EPMs provide a compact representation of the set of possible execution paths of a business process by defining information dependencies instead of the order of activities. The flexibility that EPMs provide is utilized by intelligent agents managing the execution with the objective to optimize the Key Performance Indicators (KPIs) under consideration of the operating conditions. This paper demonstrates the practical application method of the iEPM approach as intelligent BPM engine where agents autonomously adapt their behavior in accordance to the current operating conditions for optimizing KPIs. The advantages of this method are discussed and statistically analyzed using a simulation based approach and the business process "new customer" found in banking.

Keywords: Business Process Modeling, Process Flexibility, Optimization, Relational Reinforcement Learning, Particle Swarm Optimization.

1 Introduction

Business process automation comprises the creation of a process model and the execution of this model during runtime by a BPM engine. The process models do not necessarily provide the required flexibility for dealing with changing operating conditions. In particular, process designs may be optimal in certain operating conditions only. If changes occur, it may be necessary to adapt the process model or process instances in order to handle the new situation accordingly.

In [1] we introduced the Executable Product Model (EPM) for modeling business processes. The EPM contains a compact representation of the set of possible execution paths of a business process by defining information relationships instead of task sequences as in process models. The different execution paths can be executed independently from each other which allows to execute the activities of different execution paths either in a sequence or in parallel. In a process model, such behavior has to be modeled explicitly. We take advantage of the flexibility provided by the EPM during runtime by using a Multi-Agent System (MAS). A MAS was chosen, as it is capable of solving complex tasks in

distributed environments and has been applied successfully in practice. In combination with self-adaptation they are a powerful tool. Self-adaptation is the ability of a software system to adapt to dynamic and changing operating conditions autonomously [2]. The MAS manages the execution of EPMs in order to optimize the KPIs. In [3,4] we introduced a learning mechanism for this task based on two combined machine learning approaches, Relational Reinforcement Learning (RRL) with Genetic Algorithm. We successfully showed how the agents can learn in a static environment.

In this paper we evaluate the advantages of the iEPM approach applied as intelligent BPM engine on the basis of a “new customer” business process. Particularly, the limitation of using regular process models in changing scenarios is shown. Several simulation based experiments are conducted in order to show how the agents are capable of optimizing the KPIs by adapting their behaviour autonomously in changing scenarios.

The paper is outlined as follows: In Sect. 2 the elements of an EPM are explained. Moreover, the requirements of the “new customer” business process and the corresponding EPM are described. The developed iEPM approach and its application are described in Sect. 3. In Sect. 4 the conducted experiments are analyzed. Related work is discussed in Sect. 5. A conclusion and an outlook to future work end the paper.

2 The Executable Product Model

In [1], the EPM was introduced which is based on the Product Data Model (PDM). PDMs were introduced as part of a process design methodology termed product-driven workflow design [5]. Both models have been developed specifically for the information-intensive service industry. Similar to the PDM, an EPM comprises all information that accrues during the execution of a business process. Information nodes and their dependencies are the main elements of an EPM.

The information nodes correspond to abstract information like application data, business objects, documents, decisions which are part of the business process. EPMs can be modeled on different levels of abstraction as an information node may be a single variable or a business object. Information nodes are visualized as circles. An EPM has exactly one root information node representing the final result of the business process (e.g. the outcome of an insurance claim or credit application). The dependencies between information nodes determine which information must be available before another information can be generated, e.g. a decision can only be made if the application form has been filled out correctly. These dependencies reflect the production or execution order of the information elements and are modeled as directed arcs. These arcs represent production rules describing which task has to be performed to create a new value stored in the information node the corresponding arc is pointing at. Thus, production rules are similar to activities in a business process. The information node(s) that the production rule depends on are called origin nodes, the created node is the destination node of the production rule. Each information node is the

destination node of at least one production rule. As leaf nodes do not depend on any other node, they have special production rules that can be executed at any time - visualized as dashed arcs. Two kinds of dependencies can be specified. In an AND dependency all origin nodes of the related production rule must have been created before the production rule can become executable. By means of an OR dependency, alternative execution paths are specified which can be executed independently. As production rules are processed by resources (machines or humans), a role is assigned to each production rule. A role describes what skill or position a resource must have in order to execute the production rule. Additionally, a constraint can be attached to each production rule that determines under what circumstances this production rule can be executed (depicted as square brackets). In the following, an example of an EPM is provided based on a “new customer” business process.

The “new customer” process is implemented in a large German bank. For this paper it was simplified and made anonymously. The process is initiated when a new customer applies for one of the financial products the bank offers, e.g. a new customer intends to open a debit account. Based on the business requirements, several process designs are feasible. Due to space restrictions, only two options are considered and analyzed. In option A, the activities “Check Financial Situation”, “Check Application” and “Check Customer History” are modeled in parallel (see Fig. 1). In option B these checks are modeled in a sequence as listed above. The parallel design has the advantage that it can lead to a better cycle time than the sequential design. The disadvantage of the parallel design is that certain tasks may be executed unnecessarily. This is the case, when the application is rejected due to a negative financial situation, while the customer history and the correctness of the application are still being checked. Other design options are retrieved if the order of the checks is switched.

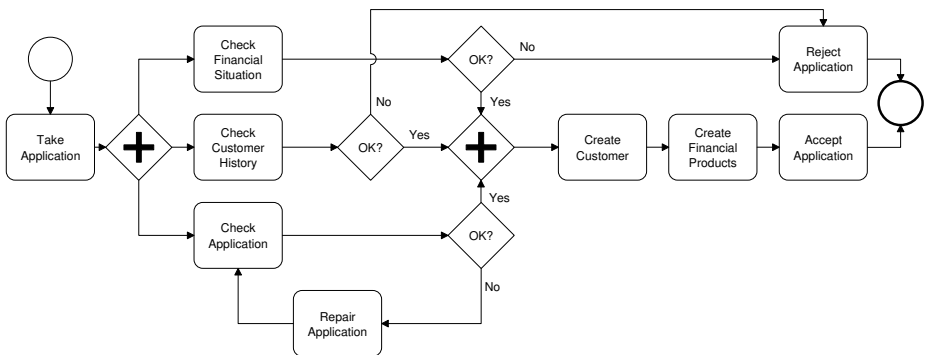


Fig. 1. The parallel design of the “New Customer” business process as BPMN model

For creating the EPM, the information elements of the business process must be identified and the dependencies between them must be specified. The resulting EPM is depicted in Fig. 2.

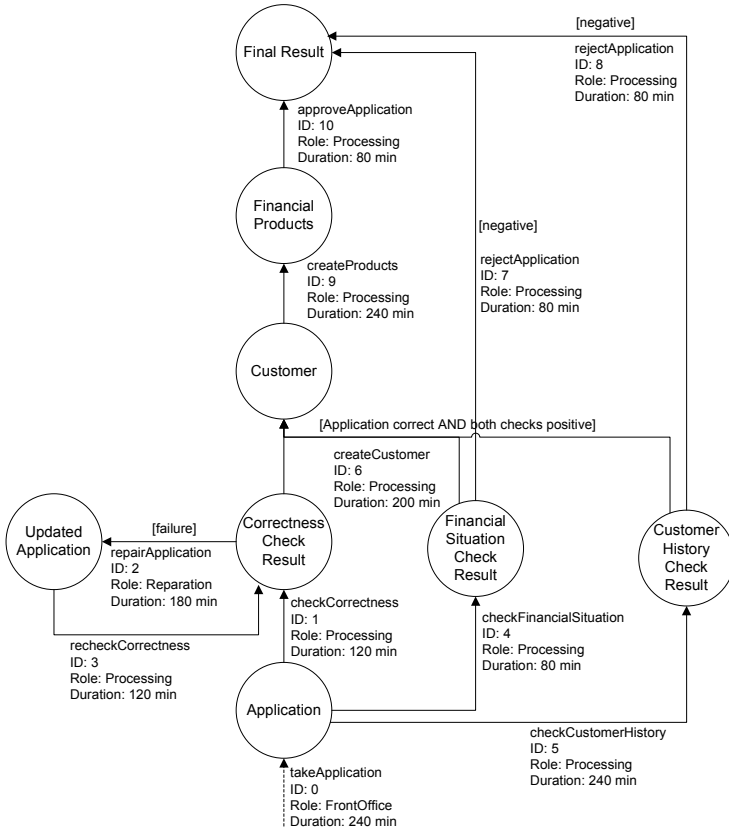


Fig. 2. Executable Product Model “New Customer”. Production rule with ID 6 is an AND dependency with three origin nodes, whereas the production rules with IDs 7, 8 and 10 are an example of an OR dependency (there are three different production rules for creating root node “Final Result”).

The EPM comprises different variants, among them also the execution paths corresponding to the two design options, e.g. the paths for approval without repair actions:

- Variant 1: $0 \rightarrow \{1, 4 \text{ and } 5 \text{ in parallel}\} \rightarrow 6 \rightarrow 9 \rightarrow 10$
- Variant 2: $0 \rightarrow 4 \rightarrow 1 \rightarrow 5 \rightarrow 6 \rightarrow 9 \rightarrow 10$

As one can see, the EPM comprises the execution paths of the two different aforementioned process designs. By modeling information dependencies, the EPM does not fix the actual execution order which enables the agents to choose the optimal path through the EPM for each instance. Moreover, the execution path can be changed anytime. This is an advantage that other approaches do not provide as they choose a specific variant at instantiation without having the possibility to change it afterwards.

3 The iEPM Approach

In this section, the iEPM approach is explained which combines a MAS with machine learning techniques.

The MAS comprises two different kinds of agents: Resource Agents (RAs) and Intelligent Business Objects (IBOs). A RA has a specific role and is modeled for each human and technical resource (e.g. web service). A new IBO is created for each execution of exactly one EPM. This is similar to a BPM engine in which a process instance is created for each process execution. The IBO determines the executable production rules by the means of an execution algorithm. Once the executable production rules are determined, it has to decide which of them should be executed and when. For each production rule that is selected for execution, the IBO negotiates the assignment with the RAs (based on negotiation protocols) and observes the execution progress. The intelligence of the IBOs stems from a hybrid machine learning approach.

The behavior of the IBO is based on Relational Reinforcement Learning (RRL). As common in RRL an agent (in our case the IBO) selects an action a^t based on its current state s^t at time t . After performing the selected action, the agent receives a reward r^{t+1} and moves to the next state s^{t+1} . The reward r^{t+1} reflects how good it was to execute action a^t in state s^t . Thus, the reward enables the agent to learn. In the iEPM approach, the learning of the appropriate actions is based on a probabilistic policy as in [6]. The probabilistic policy contains a finite set of n predefined rules. Each rule $r_i, i = 1, \dots, n$ consists of a condition c_i , an action a_i , and a probability p_i . To select an action based on this policy the condition part of each rule is consecutively evaluated until one evaluates to true. If the condition is fulfilled, the corresponding action is executed based on the assigned probability. If the action is not executed the next rule in the policy is tested. In our MAS, all IBOs have the same manually created policy and use the above described policy iteration to determine their actions. Each IBO uses a PROLOG engine to represent its state. PROLOG queries are executed for determining whether a policy condition is fulfilled. The probabilities of the policy must be determined for each single scenario using an appropriate heuristic in an offline learning phase. We apply Particle Swarm Optimization (PSO) for determining the probability vector $\mathbf{P}$. PSO contains a population of particles which explores the search space by moving with particular velocities towards the optimum. The velocity of each particle is influenced by a social impact coming from the population and the individual experience of the particle (for further details see e.g. [7]). Each particle represents a solution of the optimization problem and is evaluated by a fitness function. In our optimization problem, a particle represents a probability vector and the fitness function is based on the KPIs to be optimized. In order to calculate the fitness value of a particle, a simulation is carried out. The simulation calculates KPIs such as cycle time, throughput, costs, among others. Fig. 3 illustrates the practical application method.

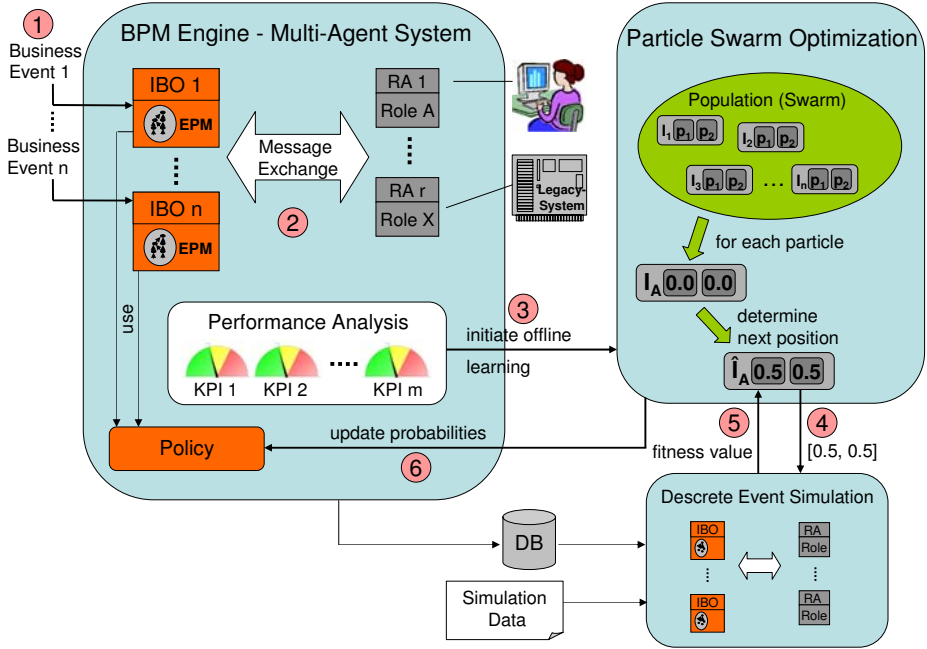


Fig. 3. The iEPM approach applied as intelligent BPM engine. (1) Business events trigger the creation of IBOs and the execution of the corresponding EPM. (2) IBOs and RAs collaborate in order to execute the EPMs. During the execution, operational data is stored in a database and the defined KPIs are calculated. (3) If necessary, the offline learning phase is triggered. PSO is illustrated on the basis of a two dimensional vector. (4) For each evaluation of a particle, a discrete event simulation is carried out. The probability vector as well as operational and simulation data are provided as input. (5) Result of this simulation is the fitness value. (6) The policy is updated with the optimal probability vector found by PSO. As one can observe, there are actually two MASs, one for the real execution and another one for the offline learning phase. The application is implemented in Java making use of the SWI-Prolog and JSwarm-PSO packages.

Our approach can be used in the BPM context as follows: The intelligent agents can act as a BPM engine and control autonomously the execution flow of business processes. The offline learning phase allows the agents to adapt their behaviour to changing situations. The learning phase can be initiated either periodically or on the basis of a defined threshold and runs in parallel to the regular execution. Once the probability vector is determined, the probabilities are updated and the agents act on the basis of the new values. The learning phase requires simulation data such as durations and arrival rates. The simulation data can be automatically calculated based on the latest production data. In our current prototype, the simulation data is provided manually. Additionally, the offline learning phase can be utilized for what-if analyses as well, e.g. to conduct business forecasts.

4 Experiments

4.1 Simulation Approach and Experimental Setup

In order to conduct experiments with the MAS, a simulation approach is required. For this work, the discrete event simulation approach [8, pp. 6-11] was chosen. A simulation component “creates” new product model instances entering the system using an exponentially distributed or constant arrival process. It also determines the task processing times based on a gamma distribution or constant values. A scenario pertains to a specific experimental setting and comprises EPM, arrival rate, available RAs, among others. As the execution of an EPM depends on the values of the information nodes, a set of so called execution data instances has to be specified as well. An execution data instance defines what value is created by each production rule. This allows to specify different cases. As the MAS is a distributed system, the communication consumes some time such as the negotiation between IBOs and RAs. This overhead is 4 time units for each executed production rule. The request of workload information requires additionally 2 time units. If the time units pertain to seconds and the production rule durations are in the range of minutes or hours, this overhead has no significant negative impact on the result.

The manually created policy used in the conducted experiments defines different execution strategies: Rule (1) requests work load information if it is not available and has not been requested yet. Rule (2) assures that work load information is kept up-to-date. Rule (3) executes all production rules that are executable. Rule (4) activates the variant with the shortest estimated execution duration if no other has been activated. If work load information is available, it is used in the calculation of the estimated duration. By activating a variant, only the executable production rules belonging to this variant are executed. Rule (5) assures that if the activated variant is still the one with the smallest estimated duration, its processing is continued. Rule (6) switches the variant if there is a shorter one than the activated.

4.2 Conducted Experiments

In this section, the conducted experiments with the “new customer” process are discussed. In the first experiment it is shown that a process model can become suboptimal if changes occur. In the second experiment it is shown how the iEPM approach is capable of handling such changes by adapting the behaviour of the intelligent agents. Let us assume that the objective is the minimization of the costs by assuring a specific service level agreement regarding the cycle time. Equation 1 defines the objective as follows:

$$\begin{aligned} & \text{minimize } \textit{mean}C \\ & \text{subject to: } \textit{mean}CT < 1200 \end{aligned} \tag{1}$$

where *meanC* are the average costs and *meanCT* the average cycle time which has to be less than a specific number of time units, here 1200. This objective can

be motivated by considering a cost reduction strategy with ensuring customer satisfaction at the same time. The customer should not wait too long for the decision about his application. The costs of executing a business process are determined on the basis of the processing times, one costs unit relates to one time unit. No fixed costs are considered. Thus, the costs are calculated by summing up the processing times of each finished and unfinished task. The process is analyzed based on two scenarios. Scenario I is defined as follows:

- Constant processing times as defined in Fig. 2.
- Table 1 lists the probabilities of the different cases. A probability reflects how often a case occurs.
- Number of RAs per role: Reparation: 1, FrontOffice: 1, and Processing: 3.
- Stop criteria: 1,000 executed EPMS.
- Constant arrival rate of 1000 minutes.

Table 1. The different cases

Case	Description	Probability
1	Accept application without repair	0.6
2	Accept application with one repair action	0.2
3	Reject application due to negative financial situation	0.1
4	Reject application due to negative customer history	0.1

In scenario II, the situation is analyzed in which the completion of activity “Check Customer History” is delayed. There can be various causes for such a delay: the occurrence of unexpected quality problems that lead to rework or if new employees must be taught. Let us assume that quality problems lead to a longer processing time due to rework. The processing time of this activity is changed from 240 to 360 minutes.

The first experiment analyzes the two design options for each scenario. Each experimental run is repeated 20 times. The results of the simulations are listed in Table 2 which contains the mean cycle time as well as the 0.95 confidence interval. The experiments were conducted with our MAS, but similar results were received with the commercial BPM simulation tool iGrafx.

In scenario I, option B has lower costs than A. Thus, B is the preferable design. The situation in scenario II is different. As the cycle time of option B violates the

Table 2. Simulation results of experiment one

	Scenario I		Scenario II	
Design	Mean Costs	Mean Cycle Time	Mean Costs	Mean Cycle Time
Option A	1161.2 $\pm$ 2.3	931.0 $\pm$ 2.4	1271.7 $\pm$ 3.4	1016.1 $\pm$ 6.9
Option B	1116.6 $\pm$ 4.8	1117.6 $\pm$ 4.8	1232.8 $\pm$ 6.8	1233.8 $\pm$ 6.9

constraint, option B is no longer the optimal process design. Therefore option A is preferable in scenario II as it satisfies the constraint regarding the cycle time. Thus far, it has been shown that depending on the scenario, different process designs can be optimal.

In the second experiment, the situation is analyzed, in which the processing time of activity “Check Customer History” changes as described above in the middle of the simulation run. This means, that in the first half of the simulation, the setting of scenario I is used and in the second half the setting of scenario II. Fig. 4 shows the results of this experiment.

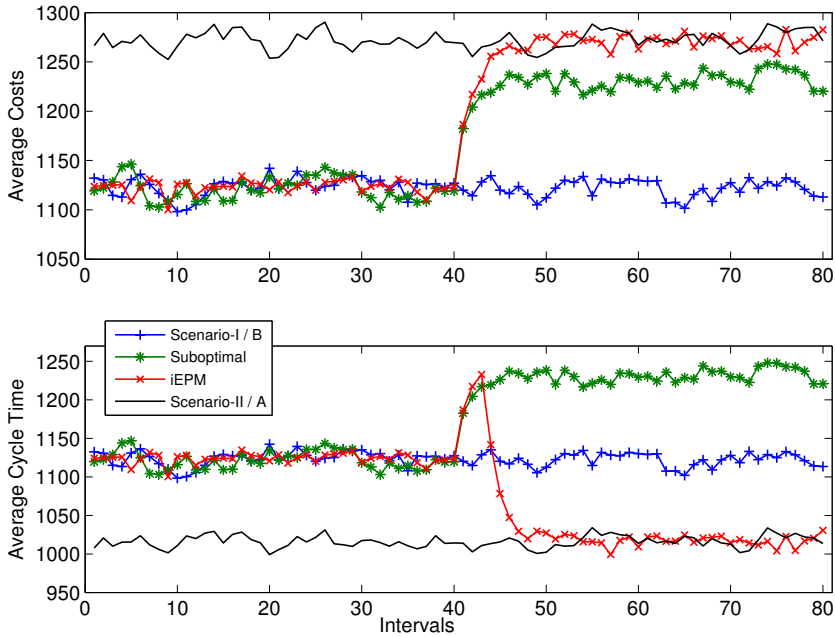


Fig. 4. Results of the different approaches. The upper figure depicts the average costs and the lower one depicts the average cycle time.

The KPIs “average cycle time” and “average costs” are calculated in fixed intervals. Run “Scenario-I / B” means that process design B is used in scenario I. Accordingly, run “Scenario-II / A” means that process design A is used in scenario II. These runs are included for comparison purposes. Run “Suboptimal” means that option B is used, but the situation changes from scenario I to scenario II in the middle of the simulation. As one can see, the cycle time increases and violates the constraint. Particularly, this shows the limitation of regular process models. Run “iEPM” shows the result of the iEPM approach. The first time that the offline learning takes place is before the simulation is started. The next time offline learning takes place is at interval 42 as the constraint regarding the mean cycle time was violated for the first time. The probabilities of the policy were

updated at the beginning of the next interval. As one can see, the agents have autonomously adapted their behaviour in accordance to the changed scenario. First, they learned to execute the checks in a sequence (by applying rule (4) and (5)) and after the second learning phase to execute them in parallel (rule (3)).

Note that the iEPM approach can cope with other scenario changes. Also, other KPIs can be optimized. Let us assume that only the cycle time is optimized and a lower arrival rate is selected in such a way that waiting times occur but the RAs are still able to execute the arriving EPMs. In this case, the parallel design is optimal. But if one of the RAs with role “Processing” is not available anymore (e.g. due to illness) the parallel design can become suboptimal. This is caused by the disadvantage of the parallel design (see Sect. 2). In experiments, the advantage of the iEPM approach could be shown analogously for this scenario change.

The conducted experiments show that the flexibility of EPMs allows the autonomous optimization of KPIs in changing operating conditions. In particular, the iEPM approach overcomes the limitation of regular process models. Note, that the learning mechanism could be integrated in non-MAS environments as well, e.g. in a centralized BPM engine. But this would require an alternative scheduling approach. Moreover, alternative modeling languages could only be used if they provide the same degree of flexibility as EPMs.

5 Related Work

There exists one approach in which swarm intelligence is applied in the area of BPM [9]. But the approach has another focus as swarm intelligence is applied for dynamic task assignment in ad-hoc processes.

Several approaches exist that focus on the flexibility of business processes. The approach described in [10] makes it possible to model and manage process variants on the basis of process family engineering techniques in an efficient way. But as the actual variant is chosen on instantiation of the business process there is no possibility to change the variant during runtime. The EPM allows to switch the variant during runtime. Further approaches improve the flexibility by allowing dynamic changes on workflow instance or schema level during runtime. The approaches mainly give an overview about the different kind of changes and how they affect the instances and schemata. For an overview of these adaptive and dynamic workflow approaches see e.g. [11]. Our approach differs from these approaches as we create a fixed model, whose flexibility is utilized by intelligent agents, controlling the process flow automatically. The adaptive workflow approaches could be used to extend our approach to further improve the flexibility of our model. In [12] a set of change patterns is introduced which is the basis for an evaluation of industry and academic approaches. Regarding our approach the late binding of process fragments pattern is of interest. Process fragments can be selected during runtime. It differs from our approach as these process fragments are selected manually or based on fixed rules. In [13] a constrained-based workflow method is introduced for providing flexibility by using a declarative

style of modeling. The model comprises a set of optional and mandatory constraints. This is well suited for variants handling and exception handling by giving the user more choices regarding the actual execution order of activities. An automatic optimization as in our approach is not integrated so far.

One of the core features of the case handling approach described in [14] is very similar to our approach. We also decide about the activation of activities based on the information available instead of the activities already executed. But the approach focuses more on the user by providing all case information and a flexible way of data entry.

The product-driven workflow approaches in [15,16] use life cycles of objects instead of product structures. As these life cycles are either used for compliance checks or for managing large process structures, the approaches are more related to a design methodology. The most related product-driven workflow approach is introduced in [17], which directly executes Product Data Models, but does not integrate machine learning in order to autonomously optimize the process execution. The authors state that their presented execution strategies may not necessarily lead to the best overall execution path. Furthermore, it is not mentioned how iterations are handled which are fundamental in business processes.

6 Conclusion and Future Work

In this paper the iEPM approach was introduced. It was shown how this approach can be used as intelligent BPM engine. Based on a simplified business process, the advantage of the iEPM approach was discussed. It was explained why EPMS provide more flexibility compared to regular process models. In experiments it was shown that the intelligent agents are capable of utilizing this flexibility in order to optimize autonomously the business process execution in changing scenarios. Thereby, the iEPM approach overcomes the limitation of regular process models. In our current prototype, the simulation data for the offline learning phase is provided manually. In the future this will be changed so simulation data can be automatically retrieved based on the production data of the executed EPMS, e.g. such as in [18].

References

1. Kress, M., Melcher, J., Seese, D.: Introducing executable product models for the service industry. In: Proceedings of the 40th Annual Hawaii International Conference on System Sciences, HICSS 2007, Waikoloa, Hawaii, January 3-6 (2007)
2. Weyns, D., Hovoeet, T.: An architectural strategy for self-adapting systems. In: International Workshop on Software Engineering for Adaptive and Self-Managing Systems, p. 3(2007)
3. Kress, M., Seese, D.: Executable product models – the intelligent way. In: Proceedings of the International Conference on Systems, Man, and Cybernetics (SMC 2007), Montreal, Canada, October 7-10 (2007)

4. Kress, M., Seese, D.: Flexibility enhancements in bpm by applying executable product models and intelligent agents. In: Business Process and Services Computing (BPSC 2007), Leipzig, Germany, pp. 93–104 (2007)
5. van der Aalst, W.M.P., Reijers, H.A., Limam, S.: Product-driven workflow design. In: Proceedings of the 6th International Conference on Computer Supported Cooperative Work in Design, London, Ont., Canada, July 12–14, pp. 397–402 (2001)
6. Itoh, H., Nakamura, K.: Learning to learn and plan by relational reinforcement learning. In: Proceedings Workshop on Relational Reinforcement Learning, July 8 (2004)
7. Kennedy, J., Eberhart, R.: Particle swarm optimization. In: Proceedings of IEEE International Conference on Neural Networks, Perth, Australia, vol. 4, pp. 1942–1948 (1995)
8. Law, A.M., Kelton, W.D.: Simulation Modeling and Analysis, 3rd edn. McGraw-Hill, Boston (2000)
9. Reijers, H.A., Jansen-Vullers, M.H., zur Muehlen, M., Appl, W.: Workflow Management Systems + Swarm Intelligence = Dynamic Task Assignment for Emergency Management Applications. In: Alonso, G., Dadam, P., Rosemann, M. (eds.) BPM 2007. LNCS, vol. 4714, pp. 125–140. Springer, Heidelberg (2007)
10. Schnieders, A., Puhlmann, F.: Variability mechanisms in e-business process families. In: 9th International Conference on Business Information Systems (BIS 2006), pp. 583–601 (2006)
11. Rinderle, S., Reichert, M., Dadam, P.: Correctness criteria for dynamic changes in workflow systems - a survey. *Data & Knowledge Engineering* 50, 9–34 (2004)
12. Weber, B., Rinderle, S., Reichert, M.: Change patterns and change support features in process-aware information systems. In: Krogstie, J., Opdahl, A.L., Sindre, G. (eds.) CAiSE 2007 and WES 2007. LNCS, vol. 4495, pp. 574–588. Springer, Heidelberg (2007)
13. Pesic, M., Schonenberg, M.H., Sidorova, N., van der Aalst, W.M.P.: Constraint-based workflow models: Change made easy. In: Meersman, R., Tari, Z. (eds.) OTM 2007, Part I. LNCS, vol. 4803, pp. 77–94. Springer, Heidelberg (2007)
14. van der Aalst, W.M.P., Weske, M., Grünbauer, D.: Case handling: a new paradigm for business process support. *Data & Knowledge Engineering* 53, 129–162 (2005)
15. Küster, J., Ryndina, K., Gall, H.: Generation of business process models for object life cycle compliance. In: Alonso, G., Dadam, P., Rosemann, M. (eds.) BPM 2007. LNCS, vol. 4714, pp. 165–181. Springer, Heidelberg (2007)
16. Müller, D., Reichert, M., Herbst, J.: Data-driven modeling and coordination of large process structures. In: Meersman, R., Tari, Z. (eds.) OTM 2007, Part I. LNCS, vol. 4803, pp. 131–149. Springer, Heidelberg (2007)
17. Vanderfeesten, I.T.P., Reijers, H.A., van der Aalst, W.M.P.: Product based workflow support: Dynamic workflow execution. In: Bellahsene, Z., Léonard, M. (eds.) CAiSE 2008. LNCS, vol. 5074, pp. 571–574. Springer, Heidelberg (2008)
18. Rozinat, A., Wynn, M., Aalst, W., Hofstede, A., Fidge, C.: Workflow simulation for operational decision support using design, historic and state information. In: Dumas, M., Reichert, M., Shan, M.-C. (eds.) BPM 2008. LNCS, vol. 5240, pp. 196–211. Springer, Heidelberg (2008)